

GoalVerse

FIFA World Cup 2026 - Companion & Prediction Platform

Technical Architecture & Data-Flow Overview

How the system is structured, where its data comes from, and how each part works together - from the schedule and live scores to the admin-driven prediction-scoring engine.

Repository: github.com/NasirSunny50/GoalVerse (private)

Stack: Flutter 3.41 / Dart 3.11 app | Dart (shelf) + SQLite backend | Docker

1. System Overview

GoalVerse is a premium Flutter companion app for the FIFA World Cup 2026 with a built-in **prediction competition** ("Compete"). It pairs a cross-platform mobile app with a lightweight Dart backend and a SQLite database. The system has three logical parts:

- **The app (client)** - the full tournament experience: schedule, teams, venues, live scores, standings, and the Compete UI (predictions, leaderboard, admin panel).
- **The backend (server)** - owns all Compete logic: accounts, prediction storage, scoring, leaderboards and the admin result entry. A Dart shelf HTTP server on port 8787.
- **The database** - a single SQLite file holding users, sessions, predictions and admin-entered results.

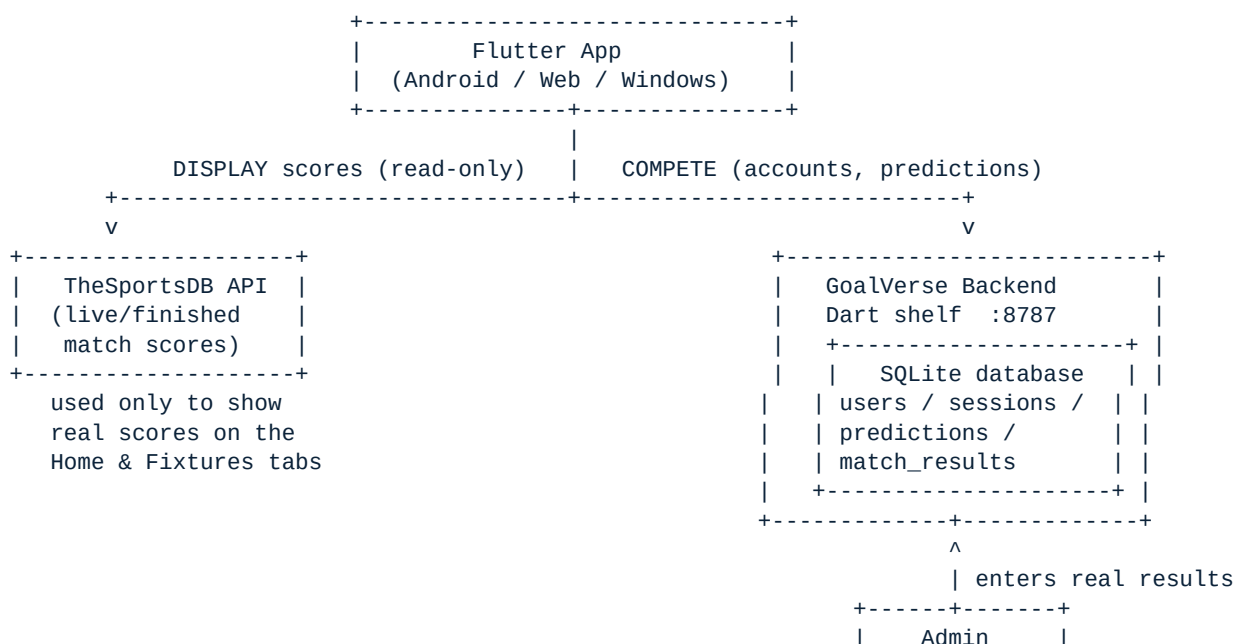
A core design principle: **the prediction competition never depends on third-party live data**. Scoring is driven exclusively by results an administrator enters, which makes points deterministic, auditable and immune to unreliable external feeds.

Technology stack

Layer	Technology
Mobile app	Flutter 3.41 / Dart 3.11, Material 3, provider state management
App data/HTTP	http package; bundled Inter & SpaceGrotesk fonts
Backend	Dart (shelf / shelf_router), runs as a native executable
Database	SQLite (sqlite3 package), single-file storage
Live scores (display only)	TheSportsDB free API (league 4429)
Deployment	Docker (multi-stage), docker compose

2. High-Level Architecture

The app talks to two independent data sources, and they serve very different purposes:



| (in-app panel) |
+-----+

The left path (TheSportsDB) is **cosmetic** - it only paints real live scores onto the schedule screens. The right path (the backend) is the **system of record** for the competition. The two never mix: a prediction is graded only against what the admin records.

3. Where the Data Comes From

Three distinct data sources feed the system, each with a clear role:

Source	What it provides	Used for
Static schedule (in-app)	The real 104-fixture WC 2026 schedule - groups, teams, venues, kick-off dates/times (schedule_data.dart).	The single source of truth for the fixture list, everywhere.
TheSportsDB (external API)	Real live / finished match scores and status (league 4429, eventsday.php, no key).	DISPLAY only - live scores on Home & Fixtures. NOT used for scoring.
Admin-entered results	The real outcome of each match: score, winner, first scorer, knockout penalties; plus knockout team assignments and the tournament outcome.	The ONLY source used to score predictions. Stored server-side.

Note: an earlier build simulated results deterministically from team ratings; that was fully removed. The app now shows **only real data**, and scoring uses **only admin data**.

4. End-to-End Data Flow

4.1 Building the schedule

On launch the app constructs the 104 matches from the static schedule. Each kick-off is stored as an absolute **UTC** instant (converted from the stadium's local time). For display it is shown in **Bangladesh time (UTC+6)**; status and countdowns are computed from UTC vs the live clock.

4.2 Live scores (display)

A provider polls TheSportsDB every 30 seconds, matches events to fixtures by a canonical team-name key, and overlays the real score/status onto each match. If a match has no real data yet, it simply shows as scheduled - no invented numbers. Group standings are computed from these real results.

4.3 Compete: registration & login

- Register with name, employee ID, email and password -> the server sends an OTP (demo code 123456) -> verifying the OTP creates the account and returns a bearer token.
- Passwords are stored as **salted SHA-256** hashes (never plaintext).
- The token maps to a session row; every authenticated request carries `Authorization: Bearer <token>`.

4.4 Making a prediction

For any upcoming match a user fills in independent **markets** (below) and saves. The app sends the prediction to the backend, which stores it against the user + match. Predictions **lock at kick-off**: after that the server rejects edits with HTTP 409, so a locked pick can never change.

4.5 Admin enters the result

After a match, the administrator signs in (on the same login screen, with hardcoded admin credentials) and is routed to an in-app **Admin Panel**. For each match the admin records the real outcome - final score, winner, first team to score, and (knockouts) whether it went to penalties. Over/Under and Both-Teams-To-Score are derived automatically from the score at the moment of entry and stored alongside it.

4.6 Scoring & leaderboard

Scoring is computed on demand. When the app requests stats / leaderboard / the prediction review, the server compares each stored prediction to the admin result for that match, awards points per market, and aggregates totals, rank, accuracy and streak. There is no background job and no live feed in this path - points are a pure function of (prediction, admin result).

4.7 Read-only review

Once a match is scored, users open it read-only and see, per market, whether they were right or wrong and the points earned, plus the actual answer. Correctness is computed by the **server** (same logic as scoring) and sent to the app, so the ticks always agree with the points.

5. The Scoring Engine

Every market is **independent and additive** - each correct answer adds its own points (so getting the exact score also earns the match-winner points). A perfect group-stage prediction is $10+25+12+8+8 = \mathbf{63}$ points (knockout adds up to +10).

Market	Points	Correct when...	Result source
Match Winner	10	Predicted side (home/draw/away) matches.	Admin (or derived from score)
Exact Score	25	Both predicted scores match exactly.	Admin score
First Team to Score	12	Predicted side (home/away/none) matches.	Admin
Total Goals	8	Under 3 (0-2) vs 3 or more matches.	Derived from admin score
Both Teams to Score	8	Yes/No matches.	Derived from admin score
Penalties (knockout)	10	Yes/No - did it go to a shootout.	Admin

Tournament-long markets (graded once the Final is recorded): Champion +50, Runner-up +30, Golden Boot +20, Golden Glove +20 (each predicted as a team). These are double-gated - they only pay out when the admin has recorded the outcome *and* the Final has kicked off.

6. The Backend & Database

6.1 Database schema (SQLite)

Table	Holds
users	email, name, employee_id, salted SHA-256 pw_hash, salt, created_at
sessions	bearer token -> email (login sessions)
match_predictions	email, match_id, winner, home/away score, first_scorer, over_under, btts, penalties
match_results	match_id, admin-assigned home/away team, winner, score, first_scorer, over_under, btts, penalties, confirmed_at
tournament_predictions	email, winner / runner-up / golden-boot / golden-glove team

The schedule itself is not in the DB; it is exported from the app's own engine to `data/fixtures.json` so match IDs match exactly on both sides.

6.2 Key HTTP endpoints

Method & Path	Auth	Purpose
GET /health	-	Liveness check
POST /auth/register, /auth/verify-otp, /auth/login	-	Account creation & sign-in (admin signs in here too)
GET /me, /stats	Bearer	Profile + computed stats
GET /predictions	Bearer	User's predictions + per-match review (result, points, hits)
PUT /predictions/match/<id>	Bearer	Save a match prediction (locks at kick-off)
PUT /predictions/tournament	Bearer	Save tournament picks
GET /leaderboard?period=...	optional	Ranked board (global / weekly / monthly / all-time)
GET /fixtures	-	Schedule + admin knockout teams + results overlay
GET /admin/results	Admin	Every fixture with its admin result
PUT /admin/result/<id>	Admin	Record a match result / assign knockout teams
DELETE /admin/result/<id>	Admin	Clear a match result

7. Admin System & Knockout Handling

- **Admin identity** is a hardcoded account, enforced server-side: a request is admin only if its token belongs to the admin email. The admin email cannot be registered or used to predict, and all `/admin/*` endpoints reject non-admins with 403.
- **Knockout teams:** the schedule's 32 knockout matches start as placeholders ("Winner Group A"). When the bracket is known the admin assigns the two real teams; the assignment is overlaid through `/fixtures` so the match shows real teams and becomes predictable everywhere.
- Predictions are **side-based** (home/away), so grading a knockout match is identical to a group match once its teams are set.

8. Security & Integrity

- Passwords: salted SHA-256, never stored or transmitted in plaintext.
- Locked predictions are immutable - the server rejects any edit after kick-off (HTTP 409), independent of the client.
- Admin authority is verified on the server for every admin action; the client is never trusted to declare itself admin.
- Per-market correctness and points come only from the server, so the displayed ticks can never disagree with the awarded points.
- For production, the hardcoded admin password should be overridden via the `GV_ADMIN_PASSWORD` environment variable, and the repository kept private.

9. Deployment (Docker)

The backend ships as a Docker image so it can run on any host. A multi-stage build compiles the Dart server to a native executable and runs it on a slim Debian image with the system SQLite library.

```
# from the repo root
docker compose up -d --build      # backend on port 8787, with DB + fixtures
```

```
# build the release app pointed at that backend
scripts/build-apk.sh http://<server-ip>:8787
adb install -r build/app/outputs/flutter-apk/app-release.apk
```

On first run the container auto-creates the SQLite database and seeds the schedule into a persistent volume (`goalverse-data`). The database survives container and image rebuilds; it starts empty (no accounts) with the admin built in.

10. Backup & Disaster Recovery

All competition state lives in the SQLite database inside the Docker volume. Backups protect against an accidental volume wipe, corruption or a dead host.

Scenario	Recovery action
Container / image broke	<code>docker compose up -d --build</code> (volume data intact)
Volume wiped (down -v)	<code>docker compose up -d</code> , then restore the latest backup
Database corruption	<code>PRAGMA integrity_check</code> ; if not ok, restore last good backup
Host machine died	On a new host: clone, compose up, restore an off-host backup

```
scripts/db-backup.sh                                # consistent online snapshot -> backups/
scripts/db-restore.sh backups/goalverse-<ts>.db      # restore into the running stack
```

Best practice: take a backup before each round of matches, and copy it off-machine (cloud / another disk). This full cycle - backup, simulated data loss, restore - has been verified end-to-end.